

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL3-Reverse Engineering

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192524281
Name	KAVIYA.G

COURSE OUTCOME COVERED

CO5: Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1. The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

Speaker	Message
User	I want to book a flight to Chennai next Friday.
Chatbot	Sure! What time would you like to depart, and do you have a preferred airline?
User	Morning flight, no preference.
Chatbot	I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?

Q2. The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d) classifying the

Real-Time Emotion Detection System — processing pipeline



agent type (e.g., simple reflex, model-based) with justification.

Q3. An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to reach the diagnosis, with justification, (b) identifying the expert-system-shell components illustrated (knowledge base, inference engine, explanation facility), (c) explaining how the shell would perform knowledge acquisition to add a new rule for dengue, and (d) explaining, with reasoning, whether backward chaining could reach the same diagnosis.

Rule	IF (condition)	THEN (conclusion)
R1	fever = yes AND cough = yes	suspect = flu
R2	suspect = flu AND body_ache = yes	diagnosis = Influenza
R3	fever = yes AND rash = yes	suspect = measles

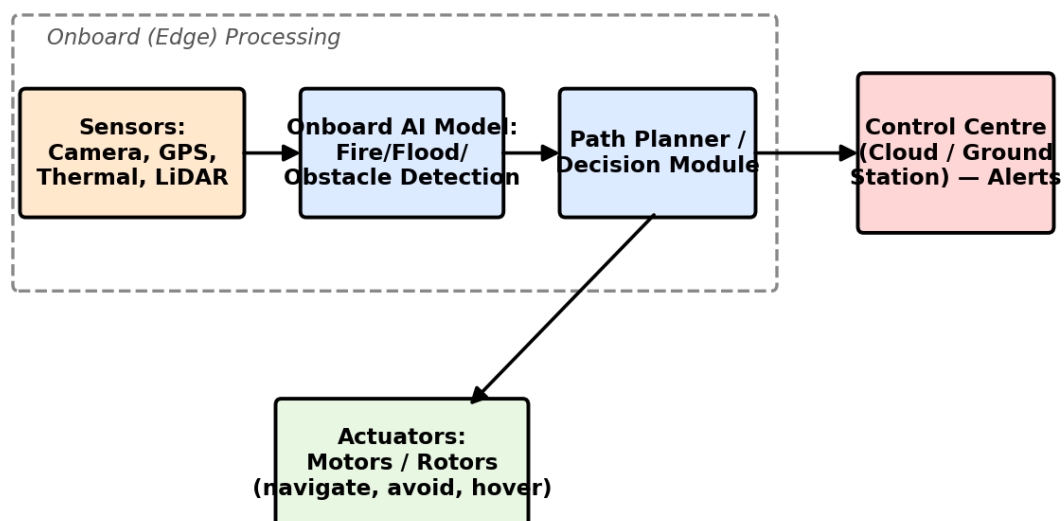
Given facts: fever = yes, cough = yes, body_ache = yes

Reasoning trace: R1 fires first (fever & cough matched) → suspect = flu. R2 then fires (suspect = flu & body_ache matched) → diagnosis = Influenza.

Explanation generated by the shell: “Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza.”

Q4. The diagram below shows the system architecture of an Autonomous Drone used for Disaster Detection with Edge AI deployment. Reverse-engineer the design by: (a) writing the PEAS description of the drone agent, (b) classifying the agent type (goal-based/utility-based) with justification, (c) identifying which components run on the edge versus the cloud and justifying why on-edge processing was chosen for detection and obstacle avoidance, and (d) identifying whether the agent's control architecture is reactive, deliberative, or hybrid, with justification.

Autonomous Drone — Disaster Detection (Edge AI) system architecture



Q5. The table below shows sample output logs from an AI-based Deepfake & Face Authenticity Verification system across five video frames. Study the output and reverse-engineer the system by: (a) identifying the likely input features used by the model (e.g., blink rate, texture/artifact score), (b) identifying the type of model/technique likely used to produce these outputs, (c) explaining how the final Real/Fake decision may have been derived from the frame-level confidence scores,

and (d) suggesting one way a swarm/ensemble-based approach could improve the system's reliability.

Frame No.	Blink Rate (per 10s)	Texture/Artifact Score	Fake Confidence (%)	Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

Overall system verdict: Fake (3 of 5 frames flagged — majority vote)

Code of Conduct:

I KAVIYA.G (192524281) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:KAVIYA.G

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Identification of Agent Type & PEAS (correctly inferred from the given artifact)	Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact.	Identifies the agent type and PEAS description correctly with only minor omissions.	Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact.	Agent type/PEAS identification is largely incorrect or not attempted.	10
Identification of Architecture & Components (mapping artifact evidence to system components)	Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact.	Identifies most components correctly with reasonable mapping to the artifact; minor gaps present.	Identifies some components; mapping to the artifact is weak or partially incorrect.	Little or no correct identification of architecture/components.	10
Identification of Underlying Technique/Algorithm/Reas	Correctly reverse-engineers the technique, algorithm, or	Correctly identifies the technique/mechanism	Identification is only partially correct or lacks	Technique/mechanism is incorrectly identified or not addressed.	10

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
oning Mechanism	reasoning mechanism (e.g., chaining type, ML technique) with clear supporting evidence.	with adequate, mostly accurate justification.	sufficient supporting evidence.		
Quality of Justification & Use of Evidence (linking conclusions back to the given artifact)	Every conclusion is clearly justified with specific, accurate references to the given artifact.	Most conclusions are justified with reasonable references to the artifact.	Justification is vague or only loosely connected to the artifact.	Conclusions are stated with little or no justification or evidence.	10
Clarity & Completeness of Presentation (addresses all sub-parts of the question in an organised manner)	All sub-parts are addressed completely, in a clear, well-organised, and logically structured manner.	Most sub-parts are addressed clearly with minor gaps in organisation or completeness.	Some sub-parts are missing or the presentation is unclear/poorly organised.	Response is incomplete, disorganised, or largely missing.	10
				Total	50

Q1. Flight-Booking Chatbot – Reverse Engineering (10 Marks)

The artifact given is a short transcript between a user and an AI flight-booking chatbot. Although only two conversational turns are shown, they contain enough behavioural evidence slot-filling, clarification questions, and a proactive suggestion — to reverse-engineer the underlying agent design fairly completely.

(a) PEAS Description

PEAS stands for Performance measure, Environment, Actuators, and Sensors, and is the standard framework used to formally specify a rational agent before deciding on its architecture. Applying it to the flight-booking chatbot:

PEAS Element	Description (evidence-based)
Performance measure	Correctly captures user intent with the fewest possible clarification turns; retrieves accurate, up-to-date flight search results; keeps the user satisfied and engaged throughout the dialogue; ultimately completes a valid, confirmed booking; responds with low latency so the conversation feels natural
Environment	A conversational text/voice interface with a single human user on one side, and a backend flight/airline reservation database (GDS) on the other; the environment is partially observable (the chatbot cannot see the user's unstated preferences, budget, or true intentions, only what is typed); it is dynamic (seat availability and prices change while the conversation is happening); it is sequential (each turn's correct response depends on everything gathered in earlier turns, not just the current message); and it is discrete (a finite set of intents, slots, and flight options)
Actuators	The mechanisms through which the chatbot acts on its environment: generating and sending text responses to the user, rendering a list of flight options on-screen, and triggering backend API calls to search or eventually book a flight
Sensors	The mechanisms through which the chatbot perceives its environment: a text/speech input parser that reads each incoming user message, and a session/context memory that lets it remember what has already been said earlier in the same conversation

Two points are worth calling out explicitly because they are easy to under-justify: first, the environment is only partially observable — the chatbot never truly knows whether the user is happy with the offered flights, it can only infer this from subsequent replies; second, the task is sequential rather than episodic, because the second chatbot response (offering the cheapest flight) is only correct because it correctly remembered the destination and date supplied one turn earlier.

(b) Agent Type Classification, With Justification

The chatbot is best classified as a Goal-Based Agent, with a light utility-based ranking behaviour layered on top of the goal-based core.

To justify this classification properly, it helps to rule out the alternative agent types one by one against the concrete evidence in the transcript:

- Not a Simple Reflex Agent: a simple reflex agent maps the current percept directly to an action using condition–action rules, with no memory of the past. But the chatbot's second reply (“I found 3 morning flights to Chennai on Friday...”) is only correct because it remembers the destination (Chennai) and date (next Friday) supplied in the very first user

message, one full turn earlier. This dependence on conversational history rules out a simple reflex design.

- Not a purely Model-Based Reflex Agent: a model-based reflex agent maintains an internal state to handle partial observability, but still only reacts to the current state without planning toward a defined future outcome. The chatbot does more than this — it is clearly working toward a specific end-state (a completed, valid booking), which is the defining feature of goal-based behaviour.
- Goal-Based evidence: the agent maintains an internal goal (“complete a valid flight booking matching destination = Chennai, date = next Friday, and any other slots the user supplies”). Because the initial message leaves the departure time and airline preference unspecified, the agent proactively asks “What time would you like to depart, and do you have a preferred airline?” — a slot-filling question that only makes sense if the agent is actively working toward completing a defined goal state, not merely reacting.
- Utility-Based trace layered on top: the final line, “Shall I show you the cheapest option first?”, implies the agent is not treating all three retrieved flights as equally good — it has already internally ranked them by a utility criterion (price) and is proposing to present the highest-utility option first. This is a small but clear utility-based decision riding on top of the goal-based dialogue structure.

Overall, the most accurate single label is “goal-based agent with utility-based ranking,” since the dominant behaviour (multi-turn slot-filling toward a booking goal) is goal-based, while the final ranking decision is a utility-based refinement.

(c) Component Mapping for Each Chatbot Response

A typical conversational-AI architecture is composed of four cooperating modules: the Natural Language Understanding (NLU) unit, which extracts intent and entities from user text; the Dialogue Manager (DM), which tracks conversational state and decides what to do next; the Knowledge Base (KB), which stores or retrieves domain data such as flights; and the Natural Language Generation (NLG) unit, which converts the Dialogue Manager's decision back into natural language. Mapping each line of the transcript onto these components:

Speaker / Message	Likely Component(s) Responsible
“I want to book a flight to Chennai next Friday.” (user)	NLU – extracts intent = book_flight and entities: destination = Chennai, date = next Friday
“Sure! What time would you like to depart, and do you have a preferred airline?”	Dialogue Manager (detects that the time and airline-preference slots are still empty and decides a clarification is needed) working together with NLG (which phrases that decision as a natural clarifying question)
“Morning flight, no preference.” (user)	NLU – fills the two outstanding slots: time = morning, airline_preference = none
“I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?”	Knowledge Base (queried by the Dialogue Manager with the now-complete slot set: destination, date, time) supplies the three matching flights; the Dialogue Manager then applies its price-ranking policy; NLG formats the count and proposes the cheapest option in natural language

It is worth noting that no single line in the transcript is produced by only one component in isolation — in a production system, the NLG module is almost always the final step that converts a Dialogue-Manager decision (which itself may depend on the Knowledge Base) into the natural-language text the user actually sees. This layered pipeline (NLU → Dialogue

Manager → Knowledge Base → Dialogue Manager → NLG) is the standard architecture behind most modern task-oriented chatbots.

(d) Enhancement Using a Learning-Agent Architecture

A classic learning agent (as defined in Russell & Norvig's agent framework) is composed of four parts working alongside the existing “performance element” (the goal-based core described above):

- Critic: evaluates how well the agent is doing against a fixed performance standard, using feedback from the environment.
- Learning Element: uses the critic's feedback to make improvements to the performance element's decision policy.
- Problem Generator: suggests actions that will lead to new, informative experiences, even if they are not the locally optimal action — i.e., it introduces exploration.

Applying this architecture to enhance the flight-booking chatbot so that it personalises itself over time instead of always defaulting to “cheapest first”:

- Critic: after each booking is completed, compares the option the chatbot suggested first (the cheapest flight) against the option the user actually finally chose, to detect whether the user overrode the suggestion (e.g., picked a costlier but preferred airline).
- Learning Element: uses this comparison, aggregated over many bookings, to update the ranking policy inside the Dialogue Manager/Knowledge-Base query — for example, learning that a specific user consistently prefers a particular airline even when it is not the cheapest, and re-weighting future “show first” suggestions accordingly.
- Problem Generator: occasionally proposes a non-default option (e.g., a slightly costlier but faster or more convenient flight) instead of always defaulting to cheapest-first, purely to keep collecting preference signals that the Learning Element can use — this is the exploration component of the learning agent.

This enhancement converts the chatbot from a fixed-policy goal-based agent into a genuine learning agent that improves its personalisation over repeated use, which is exactly the kind of concrete, architecturally-grounded enhancement the question is asking for.

Q1 - Goal-Based Chatbot Agent with Learning Enhancement

Learning-Agent Feedback Loop (Enhancement)

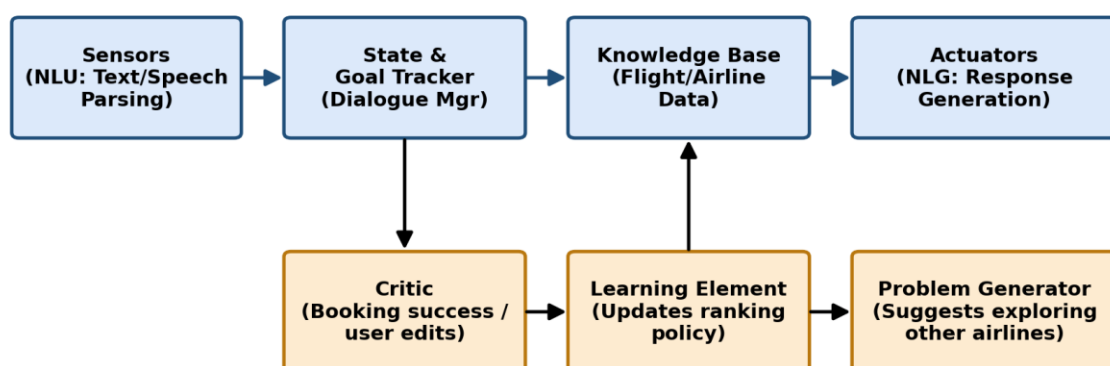


Figure 1: Goal-based chatbot core (top) enhanced with a learning-agent feedback loop (bottom).

Q2. Real-Time Emotion Detection System (10 Marks)

The artifact given is a block diagram showing a five-stage processing pipeline: Camera (Video Frames) → Face Detection → Feature Extraction (CNN) → Emotion Classifier → Output (Emotion Label + Confidence). Each stage is reverse-engineered below.

(a) Function Performed at Each Pipeline Stage

Stage	Function Performed
Camera (Video Frames)	Continuously captures the live visual scene and digitises it into a time-ordered sequence of individual frames, which forms the raw input to every later stage
Face Detection	Locates the region(s) within each frame that contain a human face and crops/isolates that region, discarding background clutter and irrelevant pixels so that later stages only process relevant facial content
Feature Extraction (CNN)	Passes the cropped facial region through convolutional and pooling layers to extract discriminative spatial features such as edges, facial-landmark positions (eyes, eyebrows, mouth corners), and local texture patterns that correlate with emotional expression
Emotion Classifier	Takes the extracted feature vector and maps it onto one of a predefined set of emotion categories (e.g., happy, sad, angry, surprised, neutral) using a trained classification layer
Output: Emotion Label + Confidence	Produces the final, human-readable prediction — the most likely emotion label — together with a numeric confidence/probability score indicating how certain the model is of that label

Read together, these five stages implement the standard perception → feature-extraction → classification → decision pipeline that almost all real-time computer-vision systems follow, and the fact that face detection happens before feature extraction is significant: it means the CNN only ever has to learn to interpret a small, face-centred region, rather than an entire, cluttered video frame, which greatly reduces the complexity of the learning problem.

(b) Likely AI/ML Technique Used at Each Stage

Stage	Likely Technique(s)
Face Detection	Classical computer-vision detectors such as the Haar Cascade / Viola–Jones algorithm, or modern deep-learning detectors such as MTCNN or SSD-based face detectors, either of which can locate a bounding box around the face in real time
Feature Extraction	A Convolutional Neural Network (stacked convolution + ReLU + pooling layers) trained on a labelled facial-emotion dataset such as FER-2013, learning hierarchical spatial features from raw pixels
Emotion Classifier	A fully-connected (dense) output layer with a Softmax activation on top of the CNN's extracted features, producing a normalised probability distribution over the emotion classes

Q2 - Emotion Detection Pipeline: Stage-wise AI/ML Technique

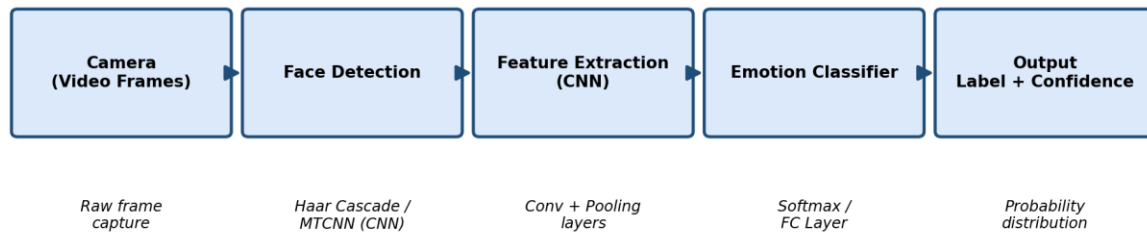


Figure 2: The five-stage pipeline annotated with the likely technique used at each stage.

(c) PEAS Description

PEAS Element	Description
Performance measure	High classification accuracy against ground-truth emotion labels; low end-to-end latency so the label updates in real time; robustness to variation in lighting, head pose, and partial occlusion
Environment	A live video feed containing a human face; the environment is partially observable, since the true internal emotional state of the person can only be inferred indirectly from visible facial expression, never observed directly; it is dynamic and continuous, since the face and lighting are constantly changing frame to frame
Actuators	The displayed emotion label and confidence score presented to a user or downstream application; in an extended system, this output could also actuate a further response (e.g., an alert or a UI change)
Sensors	The camera, which continuously captures the video frames that form the system's only percept of the world

(d) Agent Type Classification, With Justification

This system is best classified as a Model-Based Reflex Agent.

- Why not a Simple Reflex Agent: a simple reflex agent would need to map raw camera pixels directly to an emotion label using fixed condition–action rules. That is not what happens here — the system first constructs an internal representation of the world (the detected face location, and then the extracted CNN feature vector) before deciding on an output. Maintaining and using this internal representation to decide the correct current action, rather than reacting to raw percepts, is precisely the defining characteristic of a model-based reflex agent.
- Why not a Goal-Based or Utility-Based Agent: goal-based and utility-based agents reason about future states and choose actions that lead toward a desired outcome over a sequence of steps. This pipeline does not do that — each frame is classified independently based on the current percept alone, with no explicit multi-step goal being pursued and no trade-off between competing future outcomes being optimised. There is therefore no evidence of goal-directed planning or utility maximisation, ruling out the goal-based and utility-based classifications.
- Conclusion: the presence of an internal model (face location + feature vector) combined with the absence of any multi-step goal-directed reasoning places this system squarely in the model-based reflex agent category.

Q3. Medical Diagnosis Expert System Shell (10 Marks)

The artifact given is a small rule base (R1–R3), a set of input facts, and a reasoning trace showing exactly which rules fired and in what order, followed by a natural-language explanation generated by the shell. Each of these pieces of evidence is used below to reverse-engineer the shell's internal design.

(a) Type of Chaining Used, With Justification

Forward Chaining (data-driven reasoning) is used to reach the diagnosis.

The justification follows directly from the given reasoning trace: reasoning starts from the known input facts (fever = yes, cough = yes, body_ache = yes) and repeatedly matches them against the IF-conditions of the rules in the knowledge base, firing whichever rule's conditions happen to already be satisfied, and adding its THEN-conclusion as a brand-new fact. Concretely:

- R1 fires first purely because its condition (fever = yes AND cough = yes) is already satisfied by the given input facts — the system was not searching backward from a target diagnosis when this rule fired, it simply noticed that the data on hand satisfied a rule.
- Firing R1 produces the new fact suspect = flu, which is added to the working memory of known facts.
- This newly derived fact, combined with the already-known fact body_ache = yes, now satisfies R2's condition (suspect = flu AND body_ache = yes), so R2 fires next, producing diagnosis = Influenza.
- This forward, fact-to-conclusion propagation — where each newly derived fact can, in turn, trigger further rules — is exactly the mechanism that defines forward chaining, and it is fully consistent with the order in which R1 and then R2 are shown firing in the transcript.

Q3 - Forward Chaining (Data-Driven) Reasoning Trace

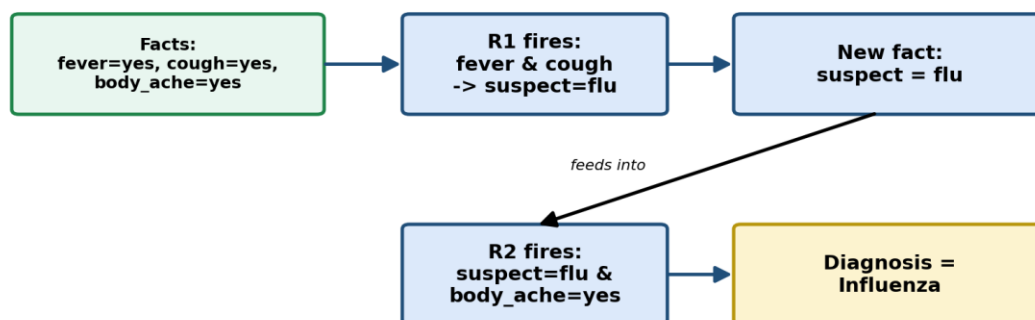


Figure 3: Forward-chaining reasoning trace — facts propagate forward into new conclusions.

(b) Expert-System-Shell Components Illustrated

An expert-system shell is conventionally decomposed into three separable components, each of which is directly visible in the given artifact:

Component	Evidence in the Transcript
Knowledge Base	The declarative rule set R1, R2, R3 (IF–THEN production rules) that encodes the domain's diagnostic knowledge, independent of how it will be executed
Inference Engine	The forward-chaining control strategy that matched the given facts against rule conditions and decided the firing order — first R1, then R2 — as recorded in the ‘reasoning trace’

Explanation Facility	The final generated sentence, “Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza,” which is a human-readable justification of exactly why the inference engine fired the rules it did, in the order it did
----------------------	---

The clean separation between the Knowledge Base (declarative ‘what is known’) and the Inference Engine (procedural ‘how to reason with what is known’) is what allows the same shell to be reused across different medical domains simply by swapping in a different rule base — this modularity is the central design principle of expert-system shells, as distinct from a hand-coded diagnostic program.

(c) Knowledge Acquisition for a New Dengue Rule

Knowledge acquisition is the process by which new domain knowledge is added to the Knowledge Base without needing to modify the Inference Engine at all, since the two are deliberately kept independent. For dengue, this process would proceed as follows:

- A domain expert (a physician) supplies the new symptom-to-diagnosis relationship that characterises dengue, for example: fever = yes, rash = yes, joint_pain = yes, and platelet_count = low.
- A knowledge engineer, or the shell's own rule-editing interface, encodes this expert knowledge as new IF–THEN production rules, in the same syntax already used by R1–R3, for example:

Rule	IF (condition)	THEN (conclusion)
R4	fever = yes AND joint_pain = yes AND platelet_count = low	suspect = dengue
R5	suspect = dengue AND rash = yes	diagnosis = Dengue

- The shell validates the newly added rules against its internal rule-syntax template (correct variable names, valid condition/conclusion structure) and checks for conflicts with existing rules — note that R3 also uses the condition ‘rash = yes’ for suspecting measles, so a priority/salience value or an additional discriminating symptom (such as platelet_count) is needed so the shell does not become ambiguous between measles and dengue when rash is present.
- Once validated, R4 and R5 are simply appended to the Knowledge Base. The existing forward-chaining Inference Engine can use them immediately on the very next consultation, with no reprogramming required — this ease of extension, achieved purely by adding declarative rules, is the single biggest practical benefit of the Knowledge-Base/Inference-Engine separation described in part (b).

(d) Could Backward Chaining Reach the Same Diagnosis? Reasoning

Yes, backward chaining could reach the identical diagnosis, although by a different reasoning path.

Backward chaining is goal-driven: instead of starting from known facts and seeing what they lead to, it starts from a hypothesis (a candidate goal) and works backward, checking whether the known facts support it. Applied to this example:

- Start with the goal to verify: “Is diagnosis = Influenza?”
- Search the Knowledge Base for a rule whose THEN-conclusion matches this goal — R2 concludes diagnosis = Influenza — and then check R2's condition: suspect = flu (not yet known, so it becomes a new sub-goal to prove) AND body_ache = yes (already a known fact, so this half of the condition is immediately satisfied).

- To resolve the sub-goal ‘suspect = flu’, again search for a rule concluding it — R1 concludes suspect = flu — and check R1's condition: fever = yes AND cough = yes, both of which are already known facts, so this sub-goal is satisfied.
- Unwinding the recursive search: since fever and cough confirm suspect = flu, and suspect = flu together with body_ache confirms diagnosis = Influenza, the original goal is proven true.
- This is exactly the same final diagnosis reached by forward chaining, arrived at via a goal-driven, top-down decomposition rather than a bottom-up, data-driven propagation. Because the rule base here is small, consistent, and has only one path connecting the given facts to the diagnosis, both chaining strategies necessarily converge on the same result — the difference between them is purely in the direction and efficiency of the search, not the final answer.

Q3 - Backward Chaining (Goal-Driven) Verification

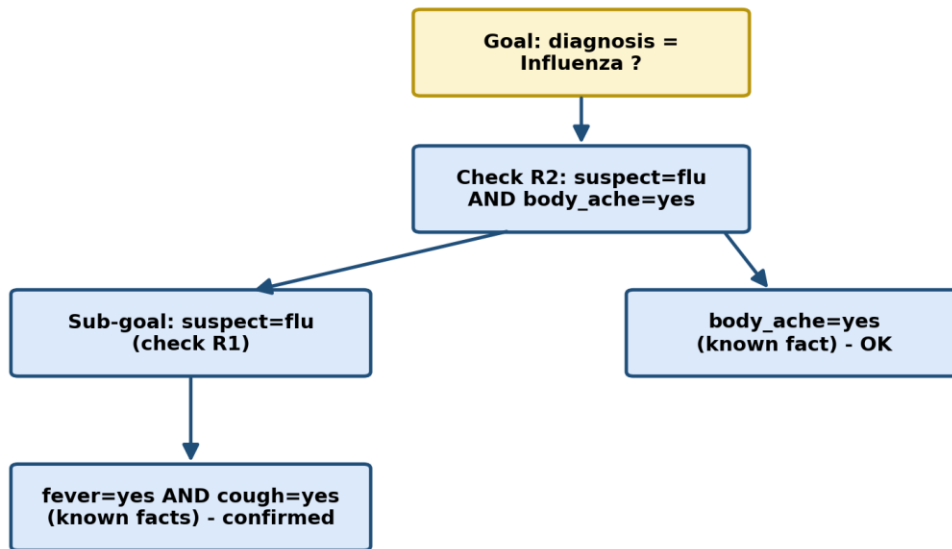


Figure 4: Backward-chaining goal decomposition, confirming the same diagnosis via a top-down search.

Q4. Autonomous Disaster-Detection Drone – Edge AI Architecture (10 Marks)

The artifact given is a system-architecture diagram showing an autonomous drone with an ‘Onboard (Edge) Processing’ boundary containing Sensors, an Onboard AI Model, and a Path Planner/Decision Module, which connects both to Actuators and to a separate Control Centre (Cloud/Ground Station). Each part of the design is reverse-engineered below.

(a) PEAS Description

PEAS Element	Description
Performance measure	Fast and accurate detection of disaster indicators (fire, flood); safe navigation with zero collisions; timely delivery of alerts to the Control Centre; efficient use of battery/energy during the mission
Environment	Outdoor disaster zones, which are partially observable (sensors give only a limited, noisy view of the true state of the disaster and surroundings); dynamic and stochastic (weather changes, GPS drift, moving obstacles, unpredictable fire/flood spread); continuous; and sequential (each navigation decision depends on the drone's evolving position and battery state)
Actuators	The motors/rotors that let the drone navigate, avoid obstacles, and hover; and the radio/data link that transmits alerts to the Control Centre

Sensors	Camera, GPS, thermal camera, and LiDAR, which together give the drone its percept of the surrounding disaster environment
---------	---

(b) Agent Type Classification, With Justification

The drone is best classified as a hybrid Goal-Based / Utility-Based agent.

- Goal-based evidence: the drone must reach and scan a target disaster zone or waypoint, which requires maintaining an explicit goal state and a route-planning capability. This kind of forward-looking route planning is a behaviour that a simple or model-based reflex agent, which only reacts to the immediate percept, cannot produce.
- Utility-based evidence: the diagram shows a dedicated ‘Path Planner / Decision Module’ that must continuously trade off several competing objectives at once — taking the shortest path to the target, maintaining a safe margin from detected obstacles, and conserving battery — which requires a utility function that can rank multiple candidate actions/paths against each other, rather than a single binary goal test.
- Combined classification: because the drone both pursues an explicit goal (reach/scan the disaster zone) and continuously optimises among competing sub-objectives while doing so, the most accurate description is a hybrid goal-based/utility-based agent, with the utility-based reasoning operating inside the broader goal-based mission structure.

(c) Edge vs. Cloud Components, With Justification

Location	Components Placed There	Why This Placement Was Chosen
Edge (onboard the drone)	Sensors (camera, GPS, thermal, LiDAR); Onboard AI Model (fire/flood/obstacle detection); Path Planner/Decision Module; Actuators (motors/rotors)	Detection and obstacle avoidance are both safety-critical and time-critical: they require millisecond-level response times that a round trip to the cloud cannot reliably guarantee, particularly since disaster zones frequently have poor or no network connectivity. Processing on the edge also avoids the need to continuously stream large volumes of raw video/LiDAR data over a limited-bandwidth link, which would otherwise be both slow and battery-expensive
Cloud / Ground Station	Control Centre – receives condensed alerts and provides human oversight/mission-level commands	Human decision-makers only need the final, distilled alert (e.g., ‘fire detected at these coordinates’), not the raw sensor stream, so this part of the system can comfortably tolerate the latency and intermittent connectivity of a cloud link

Q4 - Autonomous Drone: Edge vs Cloud Deployment (Hybrid Control)

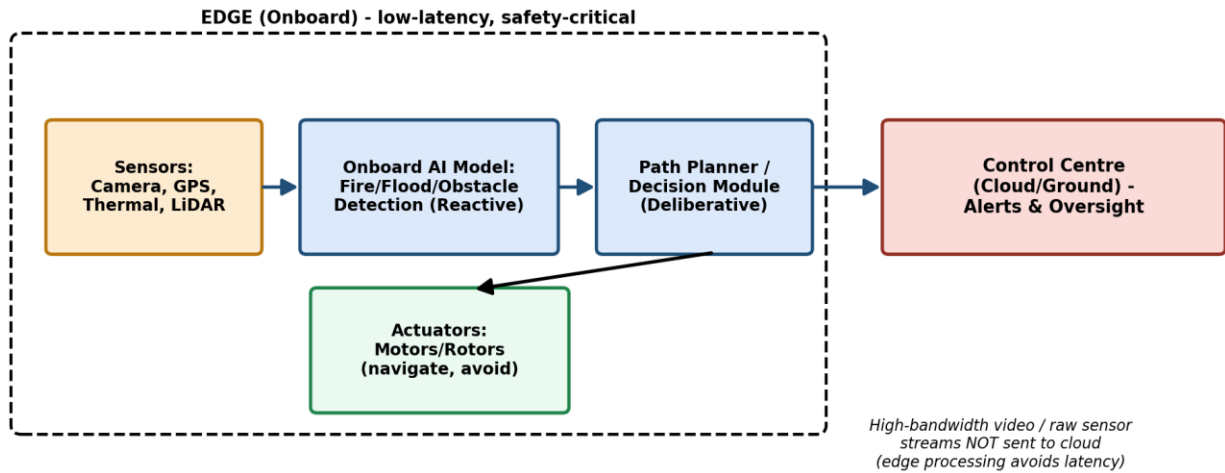


Figure 5: Edge (dashed boundary) vs. cloud placement of each architectural component.

(d) Control Architecture: Reactive, Deliberative, or Hybrid?

The drone's control architecture is **Hybrid** — it combines both a reactive layer and a deliberative layer.

- **Reactive layer:** the Onboard AI Model performs fast, low-level sensor-to-action responses for obstacle avoidance — this is a direct, near-instantaneous reflex loop that is essential for preventing collisions in real time and cannot wait for higher-level planning.
- **Deliberative layer:** the Path Planner/Decision Module reasons over the disaster information detected by the AI model, together with the drone's current goal, to plan an overall route or sequence of waypoints — a slower, higher-level reasoning process compared to the reactive obstacle-avoidance loop.
- **Justification for 'hybrid' rather than purely reactive or purely deliberative:** the diagram shows both loops feeding into the same Actuators block — the fast reactive obstacle-avoidance reflex and the slower deliberative path-planning decisions cooperate rather than operate in isolation. This layered combination of a fast reactive loop beneath a slower deliberative planning loop is the standard hybrid control architecture used in essentially all real-world autonomous robots that must operate safely in dynamic, safety-critical environments.

Q5. Deepfake & Face Authenticity Verification System (10 Marks)

The artifact given is a table of sample output logs across five video frames, recording blink rate, a texture/artifact score, a fake-confidence percentage, and a per-frame prediction, together with an overall system verdict reached by majority vote. Each of these data points is used below to reverse-engineer the underlying detection system.

(a) Likely Input Features Used by the Model

Feature	Why It Is Relevant to Deepfake Detection
Blink rate (per 10 seconds)	Natural human blinking follows a fairly consistent physiological pattern; many deepfake-generation methods fail to reproduce a realistic and temporally consistent blink frequency, so an abnormally low or irregular blink rate is a well-documented forgery cue used in several published deepfake-detection studies

Texture / artifact score	Face-swapped or GAN-generated frames very often introduce subtle blending-boundary artifacts, unnaturally smooth or inconsistent skin texture, and other micro-texture irregularities around the face that a genuine, unmanipulated camera capture would not show
--------------------------	---

(A production-grade system would typically add further implicit cues beyond these two — frame-to-frame temporal consistency, lighting/reflection mismatches on the face versus the background, and audio-visual lip-sync alignment — none of which are directly shown in this particular log, but all of which are consistent with, and commonly combined alongside, the two features that are given.)

(b) Likely Model / Technique Used to Produce These Outputs

The specific combination of a temporal cue (blink rate, which only makes sense when compared across a window of frames) and a spatial cue (texture/artifact score, computed within a single frame), each contributing to one fused per-frame confidence score, strongly suggests a supervised binary classifier built from two cooperating sub-models:

- A CNN-based spatial artifact detector, trained on labelled real-vs-manipulated face datasets (in the style of FaceForensics++ or similar benchmark datasets), responsible for producing the texture/artifact component of the score.
- A temporal sequence model, such as an LSTM or GRU (or, in more modern systems, a transformer-based temporal encoder), that tracks the blink pattern across a rolling window of frames to produce the blink-rate component of the score.
- The two streams are most likely fused — either by concatenating their intermediate features before a final classification layer, or by combining their individual scores at the decision level — into the single Fake-Confidence(%) value reported for each frame in the log.

(c) How the Final Real/Fake Decision Was Derived From Frame-Level Scores

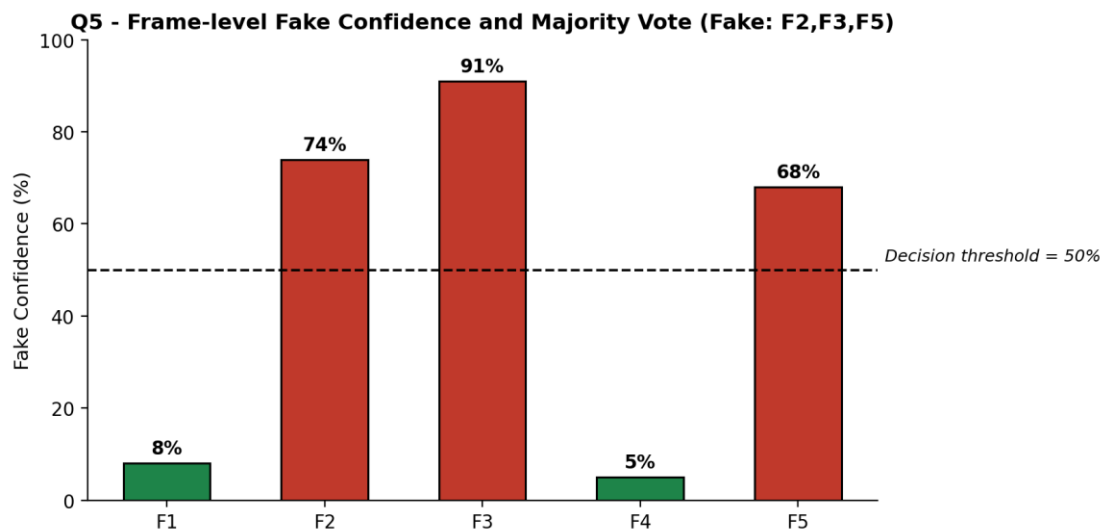


Figure 6: Per-frame fake-confidence scores against the 50% decision threshold.

Frame	Blink Rate	Texture/Artifact Score	Fake Confidence (%)	Per-Frame Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

Each frame is first thresholded independently — a Fake Confidence value above 50% is labelled ‘Fake’ for that frame, and a value at or below 50% is labelled ‘Real.’ Applying this rule to the table above: frames 2, 3, and 5 (with confidences of 74%, 91%, and 68%) cross the threshold and are labelled Fake, while frames 1 and 4 (with confidences of only 8% and 5%) stay well below it and are labelled Real. The system then aggregates these five independent, per-frame predictions using a simple majority-vote rule across the whole clip: since 3 of the 5 frames were individually flagged as Fake, the overall system verdict is Fake — exactly as stated at the bottom of the given log (“Overall system verdict: Fake — 3 of 5 frames flagged — majority vote”).

(d) Improving Reliability With a Swarm / Ensemble-Based Approach

A single per-frame majority vote from one model is fragile — it can be fooled if an adversary specifically targets the one signal (e.g., blink rate) that the model relies on most heavily. A swarm/ensemble-based approach would instead combine several independent detector ‘agents,’ each specialising in a different cue, and pool their opinions:

- Deploy multiple independent detectors as separate agents — for example, a texture/artifact CNN, a blink-rate temporal model, an audio-visual lip-sync detector, and a frequency-domain GAN-fingerprint detector — each producing its own independent Real/Fake vote for the clip.
- Fuse their outputs using confidence-weighted voting or a stacked meta-classifier, rather than a single flat majority vote, so that no single detector — and therefore no single evasion technique that fools only that one detector — can flip the overall verdict on its own.
- Add an outlier-frame rejection step that down-weights or discards frames on which the ensemble members strongly disagree with one another, reducing the influence of noisy or ambiguous frames on the final aggregated decision.

Together, these three changes make the overall system considerably more robust against adversarially crafted deepfakes that are specifically engineered to fool only one detection modality (for example, a deepfake with realistic blinking but poor texture, or vice versa), because the ensemble no longer depends on any single cue being reliable in isolation.